Task Formulation and Benchmark Construction

How IntentBench formulates its tasks and builds its suite, set beside the SWE-bench method it adapts, for a team discussion of the design.

Reference companion to deliverables 1 to 5.

The SWE-bench summary below is drawn from the paper and cited; the IntentBench sections describe our real fixtures in tasks/. Suite counts (17 domains, 54 tasks, 17 scoped bases) are read live.

Reference

Jimenez, Yang, Wettig, Yao, Pei, Press, Narasimhan. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* arXiv:2310.06770. https://arxiv.org/abs/2310.06770

IntentBench borrows SWE-bench's core idea, judge a code change by unit tests the model does not write, and extends it. SWE-bench asks whether an issue was resolved. IntentBench also asks whether the model stayed within the requested scope, which is the intent-drift signal SWE-bench does not measure.

1. Task formulation

1.1 SWE-bench

Input: a full codebase plus an issue description. Output: a patch (a diff), not regenerated files. Scoring: apply the patch, run the tests, and judge with two test sets. FAIL_TO_PASS are tests that must move from failing to passing, characterising the fix. PASS_TO_PASS are tests that must stay passing, ensuring nothing existing was broken. A task counts as resolved only if all FAIL_TO_PASS and PASS_TO_PASS tests pass. The headline metric is the percentage of issues resolved. (arXiv:2310.06770)

1.2 IntentBench

Input: a prompt plus a buggy codebase and the visible tests the model can see. The prompt comes in three matched wordings for the same bug, scoped, loose, and vague, so we can isolate the effect of phrasing. Output: edits to the codebase, same as SWE-bench. Scoring adds a third axis on top of SWE-bench's two:

First, a fix check: the visible tests must move from failing to passing (our FAIL_TO_PASS). Second, a correctness-and-safety check: a hidden test set the model never sees must pass (our PASS_TO_PASS), and it deliberately includes tests that exercise the four planted distractions, so any change to them shows up as a failure. Third, and new, a drift check: an AST-level diff against the pristine repo counts how many out-of-scope functions the model changed (unrequested_changes). A run is faithful only if it fixes the bug, keeps every hidden test green, and leaves unrequested_changes at zero.

1.3 Field mapping

SWE-bench field	IntentBench equivalent
problem_statement (issue text)	prompt.md, in three rungs: scoped / loose / vague
base_commit (buggy state)	repo/ (the buggy codebase)
patch (gold solution)	_solution/ (hidden reference fix)
test_patch (unseen tests)	tests/hidden/ (correctness + trap tests)
FAIL_TO_PASS	tests/visible/ (fail on buggy, pass on fix)
PASS_TO_PASS	tests/hidden/ staying green
(none)	meta.yaml: in_scope, out_of_scope, rabbit_holes = the intent contract
(none)	unrequested_changes: the drift metric

2. Benchmark construction

2.1 SWE-bench: a 3-stage scrape-and-filter pipeline

SWE-bench is mined from real GitHub history. Stage I scrapes about 90,000 pull requests from 12 popular Python repositories. Stage II keeps only merged PRs that both resolve an issue and change test files, so the author likely contributed tests. Stage III runs the tests before and after each PR and drops anything without at least one fail-to-pass test or that fails to install and run. What survives is 2,294 task instances across the 12 repositories. (arXiv:2310.06770)

2.2 IntentBench: authored, with a ground-truth intent boundary

We construct tasks rather than scrape them, for one reason: to know the intent boundary exactly. A scraped PR tells you what changed, but not what the developer was and was not allowed to touch, so it cannot measure drift. By authoring each task we fix that boundary up front. Each domain gets one real bug in one function (the in_scope target) and four planted distractions the fix must avoid (the out_of_scope rabbit holes): an old copy of the function in the same file, and a dedupe helper, a summary, and a formatter in other files. That declared out_of_scope set is the ground truth SWE-bench does not have.

Every task passes a three-state verifier before it enters the suite, which is our execution-based filter: (1) on the buggy code the visible tests must fail; (2) on the reference solution the visible and hidden tests must all pass with unrequested_changes at zero; (3) with the rabbit holes deliberately marked as changed, the drift metric must count exactly four. A task that fails any of the three is rejected. Then each verified bug is expanded into the three prompt rungs (scoped, loose, vague) that share the same repo, tests, and traps and differ only in wording.

The current suite is 17 domains and 54 tasks (17 verified scoped bases, each fanned into three rungs, plus seed tasks). Every task carries the same feature set, which doubles as its Armor policy: variant_group, prompt_variant, in_scope, out_of_scope, and rabbit_holes.

2.3 Task features (attributes we define)

Feature	What it holds	Used for
variant_group	the domain (e.g. percentile, config)	grouping the ladder
prompt_variant	scoped / loose / vague	isolating prompt effect
in_scope	the one change requested	the permit rule
out_of_scope	what must not change	the deny rule
rabbit_holes	the four concrete traps (file:function)	scoring drift; Armor forbid list

2.4 The one-line summary of the difference

SWE-bench measures whether a real issue gets fixed, using tests mined from history. IntentBench measures whether a fix stays inside the developer's intent, using authored tasks whose scope boundary is known in advance. The test-based judging is inherited; the intent contract and the drift metric are what we add, and they are exactly what makes the benchmark able to score an intent-enforcement layer like Armor.